

Microsoft Fabric Warehouse

Source Control & CI/CD

DBA Technical Reference | Volume 7 of 9

April 2025 | Version 1.0

Table of Contents

1.	Why Source Control Matters for Fabric Warehouse	3
2.	Git Integration Overview	3
3.	Connecting a Workspace to Git	4
4.	What Gets Stored in Git	4
5.	Branching Strategy for Fabric	5
6.	Deployment Pipelines (Dev > Test > Prod)	5
7.	CI/CD with Azure DevOps	6
8.	CI/CD with GitHub Actions	7
9.	Schema Migration Best Practices	8
10.	CI/CD Checklist	8

1. Why Source Control Matters for Fabric Warehouse

Source control is not optional for production Fabric Warehouse environments. It enables change tracking, rollback capability, team collaboration, and automated deployment — all critical for a well-governed data platform.

Benefit	Description
Change history	Every schema change is tracked with author, timestamp, and description
Rollback	Revert to any previous version of schema or stored procedures instantly
DR acceleration	Redeploy entire schema to secondary region from Git in minutes
Team collaboration	Multiple developers can work on different features using branches
Environment promotion	Controlled promotion of changes from Dev -> Test -> Production
Audit compliance	Git history provides audit trail for all schema modifications

2. Git Integration Overview

Microsoft Fabric natively supports Git integration with Azure DevOps Repos and GitHub. This allows entire Fabric workspaces to be connected to a Git repository, with workspace items serialized as structured files.

Git Provider	Support Status
Azure DevOps Repos	Generally Available (GA)
GitHub	Generally Available (GA)
GitLab	Not natively supported — use API-based automation as workaround
Bitbucket	Not natively supported

3. Connecting a Workspace to Git

- Step 1:
- In the Fabric workspace, click the three-dot menu next to the workspace name
- Step 2:
- Select Workspace settings > Git integration
- Step 3:
- Choose your Git provider: Azure DevOps or GitHub
- Step 4:
- Authenticate with your Git provider credentials
- Step 5:
- Select the Organization, Repository, and Branch to connect
- Step 6:
- Specify the Root folder within the repository for this workspace
- Step 7:
- Click Connect and sync — Fabric exports current workspace state to Git
- Step 8:
- Verify the repository contains the exported workspace item folders

INFO: Each Fabric workspace should map to a single Git branch. Use separate branches (and optionally separate workspaces) for Development, Test, and Production environments.

4. What Gets Stored in Git

Fabric Item	Stored in Git?	Format
Warehouse schema (tables, views)	Yes	JSON / SQL definition files
Stored procedures	Yes	SQL script files
Data Factory pipelines	Yes	JSON definition files
Dataflows Gen2	Yes	Mashup (Power Query) files
Notebooks	Yes	ipynb JSON format
Lakehouse	Yes	Metadata only — not data
Warehouse DATA (rows)	No	Data lives in OneLake only
Power BI reports (.pbix)	Yes (PBIP format)	Report definition files
Semantic models	Yes (PBIP format)	Model definition files

WARNING: Warehouse row data is NOT stored in Git — only schema definitions are. Data backup and recovery must be handled separately via BCDR or export processes.

5. Branching Strategy for Fabric

Branch	Purpose	Mapped Workspace
main / production	Production-ready schema and definitions	Production workspace
test / staging	Pre-production validation environment	Test workspace
develop	Active development integration branch	Development workspace
feature/xxx	Individual feature development	Developer's personal workspace or shared dev
hotfix/xxx	Emergency production fix	Branched from main, merged back after fix

6. Deployment Pipelines (Dev > Test > Prod)

Fabric includes built-in Deployment Pipelines that allow controlled promotion of workspace content across environments without manual export/import.

Step 1: In Fabric portal, create a Deployment Pipeline with 2-3 stages (Dev/Test/Prod)

Step 2: Assign a Fabric workspace to each pipeline stage

Step 3: To deploy: select the source stage and click Deploy to next stage

Step 4: Review the diff showing what has changed before confirming deployment

Step 5: Deployment Pipelines handle schema promotion — data remains in each environment

Step 6: Use deployment rules to override environment-specific settings (e.g., connection strings)

7. CI/CD with Azure DevOps

>> Azure DevOps Pipeline: Deploy Fabric Workspace on PR Merge

```
# azure-pipelines.yml
trigger:
branches:
include: [main]

pool:
vmImage: ubuntu-latest

steps:
- task: AzurePowerShell@5
  displayName: Deploy Fabric Workspace via REST API
  inputs:
    azureSubscription: $(SERVICE_CONNECTION)
    ScriptType: InlineScript
    Inline: |
      $token = (Get-AzAccessToken -Resource 'https://api.fabric.microsoft.com').Token
      $headers = @{ Authorization = "Bearer $token"; 'Content-Type' = 'application/json' }
      # Trigger Git sync on target workspace
      $body = @{ commitMessage = 'CI/CD deploy from main branch' } | ConvertTo-Json
      Invoke-RestMethod -Uri \
        'https://api.fabric.microsoft.com/v1/workspaces/$(WORKSPACE_ID)/git/updateFromGit' \
        -Method POST -Headers $headers -Body $body
    azurePowerShellVersion: LatestVersion
```

8. CI/CD with GitHub Actions

>> GitHub Actions Workflow: Auto-Deploy on Push to Main

```
# .github/workflows/fabric-deploy.yml
name: Deploy Fabric Warehouse
on:
  push:
    branches: [main]
jobs:
  deploy:
    runs-on: ubuntu-latest
    steps:
      - name: Checkout repo
        uses: actions/checkout@v3

      - name: Get Azure token
        id: token
        run: |
          TOKEN=$(curl -s -X POST \
            'https://login.microsoftonline.com/${{ secrets.TENANT_ID }}/oauth2/token' \
            -d 'grant_type=client_credentials' \
            -d 'client_id=${{ secrets.CLIENT_ID }}' \
            -d 'client_secret=${{ secrets.CLIENT_SECRET }}' \
            -d 'resource=https://api.fabric.microsoft.com' \
            | jq -r '.access_token')
          echo "::set-output name=token::$TOKEN"

      - name: Sync Fabric workspace from Git
        run: |
          curl -X POST \
            'https://api.fabric.microsoft.com/v1/workspaces/${{ secrets.WS_ID }}/git/updateFromGit' \
            -H 'Authorization: Bearer ${{ steps.token.outputs.token }}' \
            -H 'Content-Type: application/json' \
            -d '{"commitMessage": "Auto-deploy from GitHub Actions"'
```

9. Schema Migration Best Practices

- Always use CREATE OR ALTER for views and stored procedures — makes scripts idempotent and safe to re-run
- For table changes (ADD COLUMN), use ALTER TABLE statements — never DROP and recreate production tables
- Version your schema scripts sequentially (e.g., V001_create_tables.sql, V002_add_column.sql)
- Include rollback scripts alongside every migration script for emergency recovery
- Test all migration scripts on the Development workspace before promoting to Test or Production
- Document breaking changes (DROP TABLE, column rename) in the PR description and notify consumers

>> Idempotent View Creation Pattern

```
-- Safe to run multiple times without error
CREATE OR ALTER VIEW dbo.vw_SalesSummary AS
SELECT
    CAST(OrderDate AS DATE) AS SaleDate,
    SUM(SalesAmount) AS TotalSales,
    COUNT(DISTINCT CustomerKey) AS UniqueCustomers
FROM dbo.FactSales
GROUP BY CAST(OrderDate AS DATE);
```

10. CI/CD Checklist

Area	Checklist Item	Done?
Git Setup	All production workspaces connected to Git repository	[]
Git Setup	Branching strategy documented and communicated to team	[]
Git Setup	Branch protection rules on main/production branch	[]
Pipelines	CI pipeline runs on every Pull Request (schema validation)	[]
Pipelines	CD pipeline auto-deploys to Test on merge to develop branch	[]
Pipelines	CD pipeline requires manual approval to deploy to Production	[]
Scripts	All schema scripts are idempotent (CREATE OR ALTER pattern)	[]
Scripts	Rollback scripts exist for every breaking change	[]
Security	Service Principal credentials stored in pipeline secrets vault	[]
Testing	Schema deployment tested end-to-end in Test workspace	[]